

Weather Agents and MCP

Step-by-step student workbook

Author: Rajkumar Rajagobalan

Draft v1.0 | 10 September 2026 | Two-hour Google Colab workshop

One weather question. Three implementations. A clear view of who does what.

This workbook guides you from an ordinary weather API call to a model using a function tool, then to the same weather capability connected through the Model Context Protocol (MCP).

Open your lab

[Open the notebook in Google Colab](#)

[Training repository: nuvear/Agents-Basics](#)

Keep the notebook open beside this workbook. Run the cells already provided in the notebook; code shown here helps you identify or repeat the relevant operation. Preparation cells must run first.

By the end, you should be able to

- Identify the difference between a model's tool request and actual execution.
- Trace a weather result from the provider or sample through the agent.
- Identify an MCP host, client and server.
- Inspect tool discovery, arguments, execution and returned evidence.
- Explain the difference between sample data, scripted model turns and live inference.

Your details

Name: _____

Partner / group: _____

Date: _____

Working rule: a convincing answer is not execution evidence. Check the trace and the returned data.

1. Your route through the workshop

Before class: complete Sections 2 and 3. You need a browser, internet access and a Google account. This workshop uses a standard hosted Colab Python runtime. No GPU, local Python or LM Studio installation is needed.

Class time	Activity	Workbook section
0-10 min	Weather question and actors	1
10-25 min	Direct API lab	4
25-50 min	Function-calling agent lab	5-6
50-55 min	Break	-
55-70 min	MCP roles and responsibilities	7
70-100 min	MCP discovery and agent lab	8
100-113 min	Pair challenge and failure tests	9
113-120 min	Exit ticket	11

Choose your execution route

Route	Model behavior	Weather behavior
OpenAI + sample	Real model inference; API usage applies	Synthetic data; no SerpApi key needed
No-key fallback	Fixed scripted Tokyo requests; no model inference	Synthetic data; real local tool execution
Optional live demo	Real inference if OpenAI is configured	SerpApi request; separate weather key needed

The MCP connection in the fallback is real. The tool server runs in the Colab runtime. Only the model turns and weather values are simulated.

Warm-up: predict before running

The user asks: "What is the current temperature in Tokyo?"

- 1. What information could a model have learned during training?
- 2. What information must be obtained from an external source now?
- 3. What evidence would convince you that a weather tool actually ran?

Your prediction: _____

An API is an interface used by software. In this workshop, an agent is an application combining a model, tools, conversation state and a controlled execution loop. MCP standardizes communication between the host's client and a tool server.

2. Prepare Colab and the lab files

Before class | Notebook: Preparation A and Preparation B

Step 1. Open the published notebook

Use the Colab link on the cover, or open the repository and click **Open in Colab**. Sign in to your Google account if prompted. If Colab warns that the notebook comes from GitHub, check that the repository is nuvear/Agents-Basics before proceeding.

Step 2. Save your own copy

Choose **File > Save a copy in Drive**. Give the copy a name such as MCP_Weather_YourName. Work in that copy so your notes and edits are separate from the published training notebook.

Step 3. Connect a standard runtime

Click **Connect**. If choosing a runtime type, use Python with the hardware accelerator set to **None**. Wait for the connection to complete. A hosted runtime is a computer allocated to your Colab session.

Step 4. Learn how to run a cell

Click the triangular Run button beside a code cell, or select the cell and press **Shift+Enter**. Read its output before continuing. Run preparation in order: A, B, C, D. A cell that depends on an earlier definition will fail if that earlier cell has not run.

Step 5. Run Preparation A

Find **Preparation A - create the lab files** and run the collapsed setup cell. It writes the bundled scripts and sample JSON into the runtime.

Expected confirmation:

```
Workshop files ready: /content/mcp_weather_2hour
```

Open Colab's Files panel and find this folder. The setup does not extract a ZIP or mount your Google Drive. Expand the setup cell if you want to inspect the bundled source.

Step 6. Run Preparation B

Run the dependency-installation cell. Wait until it reports:

```
Dependencies installed in the isolated lab environment.
```

This creates a separate Python environment for the lab. If installation fails, stop and ask the facilitator; later cells will not repair a missing dependency.

Readiness record: files created [] dependencies installed []

Runtime files are temporary. If the runtime is deleted or replaced, repeat preparation. Save your notebook and download any files you want to keep.

3. Configure the model and check readiness

Before class | Notebook: Preparation C and Preparation D

Step 7. Add your OpenAI key, if available

Open the **Secrets** panel using the key icon in Colab's sidebar. Add a secret named exactly `OPENAI_API_KEY`, enter the key as its value, and enable notebook access. Do not paste the key into a code cell, a workbook answer or notebook output.

If you do not have a key, continue without one. The notebook will select the scripted fallback. A key's presence does not prove it has available quota or access to the selected model.

Step 8. Leave live weather optional

For the core sample labs, leave `SERPAPI_KEY` unconfigured. The optional live demonstration needs this separate `SerpApi` credential. An OpenAI key cannot authenticate the weather service.

Step 9. Run Preparation C

Leave `USE_MASKED_PROMPT = False` when using Colab Secrets. Run the credential cell. It reads the keys without printing their values and reports the selected mode.

Expected mode is one of:

```
OpenAI inference + sample weather
Scripted model fallback + sample weather
```

If Secrets is not available and the facilitator directs you to use the masked prompt, set `USE_MASKED_PROMPT = True`, rerun the cell and enter the key in the masked input. Never replace a source-code string with your real key.

Step 10. Run Preparation D

This defines `run_lab()` and `model_options()`, then checks an actual MCP connection using sample data. Look for:

```
[MCP tools/list]
[MCP tools/call] get_current_weather
```

The result should contain `mock: true` or its JSON equivalent. This verifies local MCP behavior inside Colab; it does not test OpenAI inference or live weather.

Step 11. Record your starting state

Mode: OpenAI [] scripted fallback []

MCP discovery succeeded [] sample tool result returned []

If you add a key later, rerun Preparation C before rerunning an agent lab. Use the facilitator's rehearsed model; the notebook default is `gpt-4.1-mini`.

4. Lab 1: inspect an ordinary API call

10-25 minutes | Notebook: Lab 1 - Direct API

Step 12. Predict what will happen

No model is involved. The sample run builds request parameters but reads a bundled response instead of making a network request. Predict where you will find the temperature and its unit.

Step 13. Run the first Lab 1 cell

```
run_lab("01_direct_serpapi_api.py", "--sample", "--show-raw")
```

Read the endpoint and query parameters, then the raw response. Locate answer_box. Continue to the normalized result and find temperature, location, observation_label, sources and mock.

The API endpoint shown is `https://serpapi.com/search.json`. The sample's temperature is 21 degrees Celsius. It is demonstration data, not Tokyo's current weather.

Step 14. Record the evidence

Field	Your observation
Endpoint / query, with no credential	_____
Returned location	_____
Temperature and unit	_____
Observation label	_____
Source and sample marker	_____

Step 15. Change one input: units

Predict the Fahrenheit value before running the next cell.

```
run_lab("01_direct_serpapi_api.py", "--sample", "--units", "fahrenheit")
```

Prediction: _____ Result: _____

Check: 21 degrees Celsius converts to 69.8 degrees Fahrenheit. The fixture remains the same; changing a city argument in sample mode does not fetch new weather.

Step 16. Find the live boundary

In the Files panel, open `serpapi_weather.py` and find `perform_serpapi_search`. This function sends the provider request in live mode. Do not edit it during this lab.

Explain: Why can this program fetch and normalize data without an AI model?

5. Lab 2: run the function-calling agent

25-50 minutes, first part | Notebook: Lab 2 - Function-calling agent

Step 17. Identify the actors

The application sends a question and tool schema to the model. The model requests a function with arguments. The application validates those arguments, executes the function and sends its result back to the model.

Step 18. Run the agent cell

```
run_lab("02_custom_tool_agent.py", *model_options())
```

With OpenAI configured, this makes real model requests while using synthetic weather. Without a key, it uses fixed Tokyo model requests and skips a generated final answer. The trace identifies which route ran.

Step 19. Follow the execution evidence

Find these stages in the output. A request line alone is not proof of execution.

Trace stage	What it tells you
[MODEL REQUEST]	Host starts a model turn
[TOOL REQUEST]	Model or scripted fallback requested a named tool
[VALIDATED ARGUMENTS]	Host accepted the arguments
[EXECUTION]	Python dispatches the weather function
[TOOL RESULT]	The function returned data or an error
FINAL ANSWER	Model answer, or labelled fallback completion

Step 20. Check the result and answer

Find the requested city, country code and units. Confirm mock: true in the tool result. If a real model wrote an answer, it should say the data is a sample. Compare its numbers with the tool result.

Requested tool: _____

Arguments: _____

Sample label in the result / answer: _____

Step 21. Explain the forced first call

`model_options()` uses `--force-tool` with OpenAI. The host therefore requires tool use on the first turn; the model supplies the arguments. This run is not evidence that the model independently decided to use a tool.

If there is no tool execution, the host withholds a weather answer. If the provider fails, the agent should explain the failure rather than invent conditions.

6. Lab 2: inspect the contract and loop

25-50 minutes, second part | Notebook: code tour and optional unforced cell

Step 22. Read the displayed source

Run the source-tour cell. It prints `weather_contract.py` and the short `02_custom_tool_agent.py` entry point. Find the tool name `get_current_weather` and its required arguments: `city`, `country_code` and `units`.

The schema defines acceptable input. The Python function is the implementation that does the work. Describing a tool does not execute it.

Step 23. Find validation and dispatch

Locate `execute_weather`. Notice that arguments are checked before the provider adapter runs. Find the entry point's `execute` function and identify the line calling `execute_weather`.

Which values are allowed for units? _____

What should happen to an unknown argument? _____

Step 24. Inspect the result-return path

Open `agent_core.py` in the Files panel. Find `run_loop`, then locate `role: tool` and `tool_call_id` in the tool-result message. The identifier connects the returned result to the corresponding request.

The host owns this conversation loop and limits it to four model turns and four tool requests. These limits help stop repeated calls; they do not guarantee the answer is correct.

Complete the sequence in your own words:

Question + schema -> _____

Validated arguments -> _____

Tool result + matching call ID -> _____

Step 25. Optional: let the model choose

If OpenAI is working and time remains, set `RUN_UNFORCED = True` in the notebook's optional cell and run it. This removes the requirement to call a tool on the first turn. Observe what the model actually does.

Tool used automatically? Yes ☐ No ☐ Not attempted ☐

If using the scripted fallback, skip this experiment. It cannot demonstrate model choice.

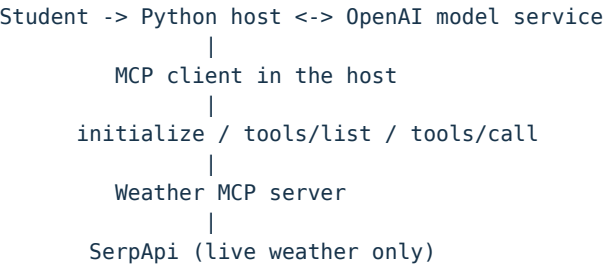
50-55 minutes: take a break

Before resuming, explain this sentence to your partner: **The model requests; application code executes; the result returns to the model.**

7. Understand the MCP boundary

55-70 minutes | Notebook: What MCP changes

Step 26. Map the components



Component	Responsibility in this workshop
Host	Conversation, model access, validation and execution budgets
MCP client	Connects the host to this server and sends protocol requests
MCP server	Advertises the weather tool and runs its implementation
Provider API	Supplies external weather-search data in live mode

Step 27. Trace discovery before execution

The client initializes its session and requests `tools/list`. The server advertises the tool name, description and input schema. The host maps that schema into the model's tool format. Later, the client sends `tools/call` with the requested arguments.

The server and client communicate through standard input/output, or **stdio**. They are processes inside the same Colab runtime. The server's standard output is reserved for protocol messages; ordinary debug prints there can break communication.

Step 28. Say what changed and what remained

In Lab 2 the host defined the schema and directly called the function. In Lab 3 the host discovers the schema and requests execution through MCP. The weather adapter still exists, now behind the server boundary. The host still owns the model conversation.

Your explanation: What does MCP standardize? _____

Colab's localhost is the hosted runtime, not your laptop. OpenAI does not connect directly to this stdio server. LM Studio remains an optional local-computer exercise outside the hosted class. MCP does not guarantee that external data or generated prose is accurate.

8. Lab 3: discover and call the MCP tool

70-100 minutes | Notebook: Lab 3 - Real MCP discovery and execution

Step 29. Read the server

Run the first Lab 3 cell. It prints `weather_mcp_server.py`. Find `@mcp.tool()`, the typed arguments and the call to `execute_weather`. This is the server exposing the weather capability.

Step 30. Inspect a real protocol exchange

The same cell runs:

```
run_lab("03_mcp_agent.py", "--inspect")
```

Find `[MCP tools/list]`. Identify `get_current_weather`, its description and parameters. The server's input schema is shown as parameters after mapping to the model tool format. Then find `[MCP tools/call]` and the sample result.

This is actual local MCP execution, even without OpenAI or SerpApi credentials.

Step 31. Connect the model loop

Run the next cell:

```
run_lab("03_mcp_agent.py", *model_options())
```

Follow discovery, model request, validated arguments, MCP invocation and result. With the no-key route, confirm the trace explicitly labels the simulated model.

Step 32. Compare responsibilities

Question	Custom function agent	MCP agent
Where does the schema come from?	_____	_____
Who manages conversation?	_____	_____
Which process runs weather code?	_____	_____
Who uses the live weather key?	_____	_____

Step 33. Show your partner the evidence

Point to the actual tool call and its returned data. A discovery result proves the tool was advertised; it does not by itself prove execution.

Discovery inspected [] tool result inspected []

Real model inference completed [] inference still pending []

9. Pair challenge and failure diagnosis

100-113 minutes | Notebook: Pair challenge and failure exercise

Step 34. Change the request

With OpenAI, run the challenge cell asking for Lisbon, Portugal in Fahrenheit. Check whether the model supplied the intended city, country and units. The sample generator is synthetic for any city; 69.8 degrees Fahrenheit is not a real Lisbon observation.

Without inference, the cell repeats the direct sample's Fahrenheit conversion. Explain the fixed fixture and unit conversion to your partner.

Step 35. Compare the evidence

Requested location and unit: _____

Returned location and unit: _____

Does the result say it is sample data? _____

What would you verify before calling it current weather? _____

Step 36. Run the failure tests

Run the notebook's test cell. It invokes the package tests in the isolated environment. The delivered version has 16 tests; look for OK at the end of the test summary. These tests do not use real provider credentials.

Step 37. Locate two boundaries

In tests/test_training.py, find the test containing invalid units in a real MCP call and the test named test_text_without_tool_withheld.

Failure	Who catches it?	What should the learner conclude?
Invalid temperature unit	_____	_____
Weather prose without a tool	_____	_____

A passing test checks the simulated or local protocol case being exercised. It does not prove that your OpenAI account works or that live weather is fresh.

Step 38. Give a one-minute explanation

Tell your partner what changes if the host uses another model service. The weather tool and server can remain the same while model connection settings change. Point out that the hosted class does not connect to your laptop's LM Studio server.

Partner's feedback: _____

10. Optional live demo and recovery guide

Instructor-led extension; use within an existing block or after class.

Optional live-weather steps

1. Add SERPAPI_KEY to Colab Secrets and grant notebook access.
2. Rerun Preparation C so the runtime receives the credential.
3. In the optional live-demo cell, change RUN_LIVE_WEATHER to True.
4. Run the cell once. It calls the direct API and, if OpenAI is enabled, the MCP agent.
5. Inspect location, temperature, units, observation label and source. Restore the flag to False after the demonstration to avoid unintended repeated calls.

Live requests consume provider quota; OpenAI inference consumes API usage. A search can return no weather answer box. Explain that failure rather than substituting a guessed value. SerpApi supplies Google's weather search result; name an upstream publisher only when the result identifies it. A retrieval timestamp is not an observation timestamp.

Recover at the first failing boundary

Symptom	Next action
LAB, run_lab or PYTHON is undefined	Run Preparation A, B, C and D in order
Package import / installation failure	Rerun B; ask the facilitator if installation fails
OpenAI mode not selected after adding key	Enable notebook access to the secret; rerun C
Model authentication / quota error	Check the account and secret privately; use fallback if unresolved
Model cannot be found	Ask the facilitator to verify the configured model ID and access
MCP startup or discovery error	Rerun D; inspect setup before attempting model inference
No usable weather result	Check city/country and returned error; do not guess
Runtime replaced or disconnected	Reconnect and rerun preparation; restore notes from saved notebook

Switch to the no-key teaching route

For an explicit fallback in an added notebook cell, after preparation:

```
run_lab("02_custom_tool_agent.py", "--offline")
run_lab("03_mcp_agent.py", "--offline")
```

No external model or weather request is made by these commands. MCP execution remains real. Record inference as pending. Stop after one unsuccessful recovery attempt during class and work with a partner so you can complete the conceptual exercise.

11. Exit ticket and your next experiment

113-120 minutes | Write first, then compare with your facilitator.

1. Does a structured tool request mean the API already ran? Explain.

2. What does MCP standardize, and what API work remains?

3. Which part changes when substituting the model backend?

4. How does sample weather with real inference differ from the scripted fallback?

5. What should the agent do if it receives no usable weather result?

Completion checklist

- ☐ I ran the direct sample and inspected its fields.
- ☐ I can explain request, validation, execution, result and answer.
- ☐ I inspected actual MCP discovery and execution.
- ☐ I can distinguish sample values from live observations.
- ☐ I completed real model inference, or recorded it as pending.
- ☐ I saved my notebook without credentials in code, notes or output.

One next experiment I would like to try: _____

References and scope

[Colab FAQ: notebooks, sharing and runtimes](#) | [Colab Secrets API](#)

[OpenAI function calling](#) | [Official MCP Python SDK v1](#)

[Facilitator guide and answer key](#)

This draft follows the repository's Colab notebook. Its 14 code cells and 16 tests were checked locally without credentials. Hosted Colab, Secrets access and live OpenAI/SerpApi execution still require instructor rehearsal. The workbook is a student practice guide; it does not certify live service availability.

Author: Rajkumar Rajagobalan